

Trabalho 1 (CI1058) – Implementação do jogo PacMan a partir da comunicação por RawSocket

André Akira A Abe Aracema

Andrieli Luci Gonçalves

Departamento de Informática - Universidade Federal do Paraná (UFPR)

aaaaa24@inf.ufpr.br alg24@inf.ufpr.br

1 Introdução

O presente trabalho apresenta a implementação em C de um *PacMan no escuro* sob arquitetura cliente-servidor via *RawSockets*. A comunicação utiliza um protocolo customizado com controle *para-e-espera* e *timeout*.

O trabalho pode ser acessado a partir deste link: <https://github.com/strawndri/redes-pacman>

2 Decisões de implementação

2.1 Timeout

Optou-se por um *timeout* baseado em **tempo de espera por ACK**. Após o envio de cada mensagem, o remetente aguarda no máximo `TIME_OUT_SEND = 100ms` por uma confirmação. Caso o ACK não chegue nesse intervalo, a mensagem é retransmitida. Para a recepção de mensagens no loop principal, também é usado um *timeout* de `TIME_OUT_GET = 100ms`, evitando bloqueio indefinido do processo.

2.2 Tipos de mensagem

Além dos tipos definidos no protocolo em sala de aula, foram adicionados os seguintes tipos:

- **MSG_DERROTA** (8): indica ao cliente que o PacMan colidiu com um fantasma e o jogo terminou com derrota;
- **MSG_VITORIA** (9): indica que o jogador coletou as 6 pastilhas e venceu;
- **MSG_FIM_RODADA** (14): sinaliza o encerramento de uma "rodada", liberando o cliente para capturar o próximo movimento.

2.3 Inicialização pelo cliente

Neste trabalho, o cliente inicia a comunicação enviando **MSG_INICIO** ao servidor. O servidor, ao

receber essa mensagem, gera e envia a visualização inicial do mapa. Essa abordagem inverte o fluxo esperado pelo enunciado (que prevê o servidor conectando o cliente).

2.4 Sistema de log

O log é gerado exclusivamente pelo servidor. A cada mensagem enviada ou recebida, uma linha é incluída ao arquivo `log.txt`. Para acompanhar o log em tempo real em um terminal separado, utiliza-se o comando:

```
tail -f log.txt
```

Cada entrada registra a ação (**ENVIU** ou **RECEBEU**), o tipo numérico da mensagem, o número de sequência, o tamanho do campo de dados e o valor do CRC. ACKs e NACKs são registrados de forma compacta na mesma linha da mensagem à qual respondem. Transferências de arquivo são registradas com a ação **ARQUIVO** e o nome do arquivo enviado.

2.5 Tratamento de 0x88 e 0x81

Os bytes `0x88` e `0x81` coincidem com marcadores de protocolo Ethernet (*VLAN tags*), podendo causar interpretação incorreta pelo *RawSocket*. Para evitar isso, sempre que um desses bytes aparece nos dados, insere-se um byte `0xFF` logo após. No receptor, o processo inverso é aplicado: ao encontrar `0x88` ou `0x81` seguido de `0xFF`, o `0xFF` é descartado. Esse mecanismo é aplicado tanto no envio de dados de arquivo quanto no envio do mapa.

Caso a adição do `0xFF` faça com que a região de dados exceda seu limite de tamanho de 31 bytes, os bytes extras são realocados para a mensagem seguinte, e assim por diante.